

INSTRUKSI TUGAS BESAR

Cloud Computing 2026

1. Deskripsi Tugas

Mahasiswa diminta membuat dan mendeploy aplikasi berbasis cloud menggunakan Google Cloud Platform (GCP). Aplikasi terdiri dari:

a. Frontend

- Frontend sudah disediakan oleh asisten.
- Mahasiswa hanya perlu melakukan clone, konfigurasi, dan deployment.

b. Backend REST API

- Backend dibuat sendiri oleh masing-masing mahasiswa.
- Backend wajib terhubung ke database MariaDB yang disediakan.
- Backend wajib menyediakan endpoint sesuai ketentuan.

c. Database MariaDB

- Database disediakan oleh asisten pada VM.
- Setiap mahasiswa menggunakan user database masing-masing.

2. Repository Frontend

Frontend sudah disediakan oleh asisten. Mahasiswa wajib melakukan clone repository berikut:

<https://github.com/Habibboys/frontend-tb-komputasi-awan-2026.git>

3. Ketentuan Deployment

Frontend dan backend wajib dideploy pada layanan GCP yang berbeda.

Layanan yang boleh digunakan:

1. Compute Engine / VM Instance (gunakan tipe e2-small, maksimal e2-standard-2; JANGAN gunakan tipe besar)

2. App Engine

3. Cloud Run

Catatan: Frontend dan backend tidak boleh menggunakan layanan deployment yang sama. Jika menggunakan Compute Engine, wajib menggunakan tipe mesin kecil seperti e2-small (rekomendasi) atau maksimal e2-standard-2. DILARANG menggunakan tipe mesin besar (e2-standard-4 ke atas, n2, c2, dsb.) untuk menghemat kuota GCP.

Ketentuan Penamaan Service:

Seluruh layanan GCP yang digunakan (Compute Engine, App Engine, maupun Cloud Run) wajib diberi nama dengan format berikut:

<code>fe-<NIM></code>	(untuk layanan Frontend)
<code>be-<NIM></code>	(untuk layanan Backend)

Contoh: `fe-2211521020` untuk frontend dan `be-2211521020` untuk backend.

4. Ketentuan Backend

Mahasiswa wajib membuat backend REST API sendiri. Backend wajib memiliki minimal satu entitas utama. Tema aplikasi dan struktur data (nama entitas, atribut, dan relasi) wajib unik per mahasiswa dilarang keras memiliki tema atau struktur data yang sama/identik dengan mahasiswa lain. Kesamaan tema maupun struktur data antar mahasiswa akan dianggap sebagai kecurangan akademik. Contoh entitas: Buku, Film, Makanan, Produk, Kendaraan

Backend wajib mendukung operasi CRUD yang dimaksud bukan CRUD biasa menggunakan array lokal, data dummy, file JSON lokal, atau data hardcoded. CRUD wajib benar-benar menggunakan database MariaDB

Contoh yang TIDAK diperbolehkan (array lokal):

<pre>const books = [{ id: 1, title: 'Contoh Buku' }];</pre>

Contoh yang TIDAK diperbolehkan (data statis):

```
app.get('/books', (req, res) => {  
  res.json([ { id: 1, title: 'Data Dummy' } ] );  
});
```

Backend harus melakukan query ke database, misalnya:

```
SELECT * FROM books;
```

atau menggunakan ORM seperti Sequelize, Prisma, Eloquent, TypeORM, dan sejenisnya.

5. Ketentuan Database MariaDB

Database MariaDB disediakan oleh asisten melalui VM. Setiap mahasiswa wajib:

1. Menggunakan user MariaDB masing-masing.
2. Membuat database masing-masing.
3. Membuat minimal satu tabel utama.
4. Menghubungkan backend ke database tersebut.

Contoh format user:

```
user_2311523001
```

Contoh format database:

```
db_2311523001
```

Kredensial Akses Database:

```
user      = root  
password = KomputasiAwan2026!
```

Catatan: Gunakan kredensial di atas untuk koneksi ke database MariaDB yang disediakan asisten. Jangan mengubah password ini.

6. Endpoint Wajib Backend

Backend wajib menyediakan endpoint berikut:

```
GET    /health
GET    /schema
GET    /items
GET    /items/:id
POST   /items
PUT    /items/:id
DELETE /items/:id
```

Endpoint `/items` boleh diganti sesuai tema aplikasi. Contoh jika tema aplikasi adalah buku:

```
GET    /books
GET    /books/:id
POST   /books
PUT    /books/:id
DELETE /books/:id
```

Namun endpoint tersebut wajib dicantumkan pada response `/schema`.

7. Endpoint `/health`

Endpoint `/health` digunakan untuk mengecek status backend dan koneksi database.

Contoh response (database terhubung):

```
{
  "status": "success",
  "message": "Backend is running",
  "database": "connected",
  "student": {
    "name": "Muhammad Nouval Habibie",
    "nim": "2311523001"
  }
}
```

Contoh response (database tidak terhubung):

```
{
  "status": "error",
  "message": "Backend is running, but database is not connected",
  "database": "disconnected",
  "student": {
    "name": "Muhammad Nouval Habibie",
    "nim": "2311523001"
  }
}
```

8. Endpoint /schema

Endpoint /schema digunakan oleh frontend untuk membaca struktur data backend secara otomatis. Response /schema wajib mencantumkan:

1. Nama mahasiswa
2. NIM mahasiswa
3. Nama resource
4. Label resource
5. Daftar field
6. Daftar endpoint CRUD

Contoh response:

```
{
  "student": { "name": "Muhammad Nouval Habibie", "nim": "2311523001" },
  "resource": {
    "name": "books", "label": "Data Buku",
    "description": "Aplikasi untuk mengelola data buku"
  },
  "fields": [
    { "name": "title", "label": "Judul Buku", "type": "text", "required": true, "showInTable": true },
    { "name": "author", "label": "Penulis", "type": "text", "required":
```

```
    true, "showInTable": true },
    { "name": "year", "label": "Tahun Terbit", "type": "number",
      "required": false, "showInTable": true }
  ],
  "endpoints": {
    "list": "/books", "detail": "/books/{id}",
    "create": "/books", "update": "/books/{id}", "delete": "/books/{id}"
  }
}
```

9. Contoh Response CRUD

GET Semua Data

```
{
  "status": "success",
  "message": "Data retrieved successfully",
  "data": [{ "id": 1, "title": "Laskar Pelangi", "author": "Andrea
Hirata", "year": 2005 }]
}
```

POST Tambah Data

```
{
  "status": "success",
  "message": "Data created successfully",
  "data": { "id": 2, "title": "Bumi", "author": "Tere Liye", "year":
2014 }
}
```

PUT Ubah Data

```
{
  "status": "success",
  "message": "Data updated successfully",
  "data": { "id": 2, "title": "Bumi", "author": "Tere Liye", "year":
2014 }}
```

DELETE Hapus Data

```
{
  "status": "success",
  "message": "Data deleted successfully"
}
```

10. Ketentuan Frontend

Mahasiswa wajib:

1. Clone frontend dari repository yang disediakan.
2. Menjalankan frontend secara lokal.
3. Memasukkan Base URL backend masing-masing.
4. Memastikan frontend dapat membaca endpoint /schema.
5. Memastikan frontend dapat menjalankan CRUD dari backend.
6. Mendeploy frontend ke salah satu layanan GCP.

11. Ketentuan Pengumpulan

Mahasiswa wajib mengumpulkan:

1. Link repository GitHub backend.
2. Link hasil deployment frontend.
3. Link hasil deployment backend.
4. Laporan singkat dalam format PDF dengan nama file:
NamaLengkap_NIM_TBCC2026.pdf (contoh: Muhammad Nouval
Habibie_2311523001_TBCC2026.pdf)
5. Video presentasi singkat (Max 7 menit)

DEADLINE : 20 JUNI 2026 JAM 23.59

12. Format Laporan Singkat

A. Identitas

Nama :

NIM :

Kelas :

Tema Aplikasi :

B. Link Hasil Pengerjaan

Repository Backend :

URL Frontend (Cloud Run / App Engine / Compute Engine) :

URL Backend (Cloud Run / App Engine / Compute Engine) :

C. Daftar Endpoint API

Method	Endpoint	Keterangan
GET	/health	Mengecek status backend dan database
GET	/schema	Mengambil struktur data
GET	/items	Menampilkan seluruh data
GET	/items/:id	Menampilkan detail data
POST	/items	Menambahkan data
PUT	/items/:id	Mengubah data
DELETE	/items/:id	Menghapus data

D. Bukti Screenshot

Lampirkan screenshot:

1. Deployment frontend berhasil.
2. Deployment backend berhasil.
3. Endpoint /health berhasil diakses.
4. Endpoint /schema berhasil diakses.
5. Frontend berhasil menampilkan data dari database.
6. Frontend berhasil menambahkan data ke database.
7. Frontend berhasil mengubah data pada database.
8. Frontend berhasil menghapus data dari database.

E. Kesimpulan

Tuliskan kesimpulan singkat mengenai hasil deployment dan integrasi frontend, backend, serta database.

13. Penilaian

Komponen	Bobot
Backend REST API sesuai ketentuan	20%
Aplikasi Berjalan Baik tanpa Error	15%
Deployment backend di GCP	25%
Deployment frontend di GCP	25%
Laporan dan Video	15%